Chapter 4 — Services (`Services/crud.services.ts`)

> **File:** `src/features/react-crud-v4/Services/crud.services.ts` **One job:** One function per API endpoint. HTTP calls only. No React, no UI, no business logic.

4.1 Why a Separate Services File?

A beginner might write API calls directly inside a component:

```
// BAD — API call inside a component
function MyComponent() {
  useEffect(() => {
    fetch("/api/NZAMP/GetStudentsAttendanceSummary")
      .then(res => res.json())
      .then(data => setStudents(data));
  }, []);
}
```

Problems with this approach:

1. **Not reusable** — if two components need the same data, you duplicate the fetch call
2. **Not testable** — you cannot test the API call without rendering the component
3. **No auth** — you have to manually add the auth token every time
4. **No validation** — if the server sends wrong data, it silently corrupts your state
5. **Mixed concerns** — the component now has two jobs: fetch data AND render UI

The services file solves all of these by extracting API calls into pure async functions.

4.2 The Real File — Line by Line

```
import apiClient from "@/features/common/API/apiClient";
import { zodParse } from "@/utils/zodParse";
import {
  GetItemsResponseSchema,
  GetCategoriesResponseSchema,
  GetYearsRetrieveResponseSchema,
} from "./crud.dtos";
import type {
  GetItemsResponse,
  GetCategoriesResponse,
  GetYearsRetrieveResponse,
  CreateItemRequest,
  CreateItemResponse,
  UpdateItemRequest,
  UpdateItemResponse,
```

```
  DeleteItemRequest,
} from "./crud.dtos";
```

`import apiClient` — a pre-configured Axios instance. Axios is an HTTP client library (like `fetch` but with more features). `apiClient` is configured with:

- The base URL of the API
- The authentication token (added automatically to every request header)
- Default timeout settings
- Error interceptors

Using `apiClient` instead of raw `axios` or `fetch` means you never have to manually add the auth token. If you forget, the request returns 401 Unauthorized — a confusing error that takes time to debug.

`import { zodParse }` — a utility function that wraps `schema.parse(data)` with better error messages. It adds the function name to the error so you know exactly which API call failed.

`import type { ... }` — `import type` imports only TypeScript types, not runtime values. This is a performance optimization: the types are erased at compile time, so no runtime code is imported. Use `import type` whenever you only need a type, not a value.

The Params Type

```
export type GetItemsParams = {
  year?: string;
};
```

Why define params as a type here? Three files need to agree on the shape of the search parameters:

1. `crud.services.ts` — the function that sends them to the server
2. `crud.queries.ts` — the hook that passes them to the service
3. `useCrudPage.ts` — the hook that builds them from user input

By defining `GetItemsParams` in `services.ts` and exporting it, all three files import from the same source. If you add a new filter parameter, you update it in one place and TypeScript guides you to update the other two.

`year?: string` — optional. The user may not have selected a year. When `year` is `undefined`, Axios omits it from the query string entirely (no `?year=undefined` in the URL).

GET Functions

```
export const getItems = async (params?: GetItemsParams): Promise<GetItemsResponse>
=> {
  const response = await apiClient.get("/api/NZAMP/GetStudentsAttendanceSummary",
{ params });
```

```
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};
```

`async` — marks the function as asynchronous. An async function always returns a `Promise`. Inside an async function, you can use `await`.

`await` — pauses execution until the Promise resolves. Without `await`, `apiClient.get(...)` returns a Promise object immediately — not the actual response. With `await`, execution pauses until the server responds, then continues with the actual response.

Why does this matter? Network calls take time (100ms to 5000ms). JavaScript is single-threaded — it cannot pause and wait. `async`/`await` is syntactic sugar over Promises that makes asynchronous code look synchronous and readable.

```
// Without async/await (Promise chain — harder to read)
const getItems = (params) => {
  return apiClient.get("/api/...", { params })
    .then(response => zodParse(GetItemsResponseSchema, response.data,
"getItems"));
};

// With async/await (same logic — much more readable)
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};
```

`apiClient.get(url, { params })` — Axios GET request. The second argument is a config object. `{ params }` is shorthand for `{ params: params }`. Axios serializes the params object into a URL query string: `{ year: "10" }` → `?year=10`.

`response.data` — Axios wraps the HTTP response in an object:

- `response.status` — HTTP status code (200, 404, 500, etc.)
- `response.headers` — response headers
- `response.data` — the actual JSON body (already parsed from JSON string to JavaScript object)

`zodParse(GetItemsResponseSchema, response.data, "getItems")` — validates the response. If the data matches the schema, returns the typed data. If not, throws an error like: `"[getItems] Zod parse failed: Expected string, received null at path: StudentName"`.

The third argument `"getItems"` is the function name — it appears in the error message so you know which API call failed.

`Promise<GetItemsResponse>` — the return type. TypeScript knows this function returns a Promise that resolves to `GetItemsResponse`. The query hook can `await` it safely.

```
export const getCategories = async (): Promise<GetCategoriesResponse> => {
  const response = await apiClient.get("/api/NZAMP/GetResponseActivities");
  return zodParse(GetCategoriesResponseSchema, response.data, "getCategories");
};
```

No params — this endpoint returns all categories without filtering.

```
export const getYears = async (): Promise<GetYearsRetrieveResponse> => {
  const response = await apiClient.post("/masters/GetYearsRetrieve", {});
  return zodParse(GetYearsRetrieveResponseSchema, response.data, "getYears");
};
```

`apiClient.post(url, {})` — this endpoint uses POST even though it's a read operation (it fetches years). This is a backend design decision — some legacy endpoints use POST for reads. The empty `{}` is the required request body (the endpoint expects a body, even if empty).

POST (Mutation) Functions

```
export const createItem = async (payload: CreateItemRequest):
Promise<CreateItemResponse> => {
  const response = await apiClient.post<CreateItemResponse>
("/api/NZAMP/RecordStudentActivity", payload);
  return response.data;
};
```

`apiClient.post<CreateItemResponse>(url, payload)` — Axios POST request. `payload` is the request body (sent as JSON). The generic type `<CreateItemResponse>` tells TypeScript what type `response.data` will be — this is a compile-time hint only, Axios doesn't validate it at runtime.

Why no `zodParse` here? Write operations (create/update/delete) return simple success/error responses. We trust our own payload construction. The risk of unexpected data is much lower than for complex read responses with 27 fields.

```
export const updateItem = async (payload: UpdateItemRequest):
Promise<UpdateItemResponse> => {
  const response = await apiClient.post<UpdateItemResponse>
("/api/NZAMP/UpdateStudentActivity", payload);
  return response.data;
};
```

Note: both create and update use `POST`. The NZAMP API uses POST for all write operations (not PUT/PATCH). This is a backend convention — always check the C# controller's `[HttpGet]`/`[HttpPost]` attribute.

```
export const deleteItem = async (payload: DeleteItemRequest): Promise<void> => {
  await apiClient.post("/api/NZAMP/DeleteStudentActivity", payload);
};
```

`Promise<void>` — the delete endpoint returns nothing useful. `void` means "this function returns a Promise that resolves to nothing". We `await` it to ensure the delete completes before the caller continues.

4.3 Error Handling

You might notice there is no `try/catch` in any service function. This is intentional.

Why no try/catch in services? Errors are handled at the React Query layer (`crud.queries.ts`). When `apiClient.get(...)` throws (network error, 500 error, etc.), the error propagates up to `useQuery`/`useMutation`, which:

1. Sets `isError: true` on the query (for `useQuery`)
2. Calls `onError` callback (for `useMutation`)
3. Makes the error available as `error` in the hook's return value

Adding `try/catch` in the service would swallow the error before React Query can handle it. The service's job is only to make the HTTP call and validate the response — not to handle errors.

The exception: If you need to transform an error (e.g., extract a specific error code from the response body), you might add a `try/catch` in the service. But this is rare.

4.4 Async/Await Deep Dive

Since `async`/`await` is used everywhere, let's understand it thoroughly.

Promises

A Promise is an object that represents a value that will be available in the future (or an error if something goes wrong).

```
// A Promise is in one of three states:
// 1. Pending — the operation is in progress
// 2. Fulfilled — the operation succeeded, value is available
// 3. Rejected — the operation failed, error is available

const promise = apiClient.get("/api/...");
// promise is "pending" while the request is in flight
// promise becomes "fulfilled" when the server responds
// promise becomes "rejected" if the request fails (network error, 500, etc.)
```

Async/Await

`async`/`await` is syntactic sugar that makes Promises easier to work with:

```
// Without async/await:
function getItems() {
  return apiClient.get("/api/...")
    .then(response => response.data)
    .catch(error => { throw error; });
}

// With async/await (same logic, much cleaner):
async function getItems() {
  const response = await apiClient.get("/api/...");
  return response.data;
}
```

What Happens When You `await`

```
async function getItems() {
  console.log("1. Starting request");
  const response = await apiClient.get("/api/...");  // pauses here
  console.log("3. Got response");  // runs after server responds
  return response.data;
}

console.log("2. Function returned (Promise is pending)");
// Output order: 1, 2, 3
```

When you `await` a Promise, JavaScript:

1. Starts the async operation
2. Returns control to the caller (the function appears to return immediately)
3. When the Promise resolves, resumes execution from after the `await`

This is why the page doesn't freeze while waiting for the server — JavaScript continues running other code while the request is in flight.

Error Handling with Async/Await

```
// If the server returns a 4xx or 5xx status, Axios throws an AxiosError
async function getItems() {
  try {
    const response = await apiClient.get("/api/...");
    return response.data;
  } catch (error) {
    // error is an AxiosError
    console.error(error.response?.status); // 404, 500, etc.
    throw error; // re-throw so React Query can handle it
```

```
  }
}
```

In our codebase, we don't add `try/catch` in services — we let errors propagate to React Query.

4.5 Common Mistakes

❌ Forgetting `await`

```
// WRONG — returns a Promise, not the data
const getItems = async (params) => {
  const response = apiClient.get("/api/...", { params }); // missing await!
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
  // response is a Promise, not an Axios response — response.data is undefined
};

// CORRECT
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};
```

❌ Using `fetch` instead of `apiClient`

```
// WRONG — no auth token, no interceptors
const response = await fetch("/api/NZAMP/GetStudentsAttendanceSummary");

// CORRECT — auth token added automatically
const response = await apiClient.get("/api/NZAMP/GetStudentsAttendanceSummary");
```

❌ Skipping `zodParse` on read responses

```
// WRONG — no runtime validation, silent failures
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return response.data as GetItemsResponse; // "as" is a lie — no validation
};

// CORRECT — validated at the boundary
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};
```

❌ Putting business logic in services

```
// WRONG — service does more than one thing
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  const data = zodParse(GetItemsResponseSchema, response.data, "getItems");
  return data.filter(item => item.CurrentYear === "10"); // filtering is UI logic!
};

// CORRECT — service only fetches and validates
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};
// Filtering happens in the hook or component
```

Next: Chapter 5 — React Query